

Comparison of Dependency Length in Natural Language and Randomized Sentences

1. Motivation

Languages use wildly different word orders, yet every language must be produced and understood by the same human cognitive apparatus. One long-standing claim is that this apparatus pushes languages toward word orders that are easy to parse. When a listener hears a sentence, each word that depends on a later word must be held in working memory until its head arrives; the farther apart two syntactically related words sit in the string, the higher this memory cost (Gibson, 1998). This motivates the Dependency Length Minimization (DLM) hypothesis: languages arrange words to keep heads and dependents close (Hawkins, 1994; Gibson, 2000).

In dependency grammar, every word in a sentence (except the root) has exactly one head; the dependency length of the head–dependent edge is the linear distance between the two positions. The total dependency length of a sentence is the sum of these edge lengths. DLM makes a sharp, testable prediction: actual sentences should have shorter total dependency length than alternative word orders that preserve the same dependency tree (Futrell, Mahowald & Gibson, 2015).

This project replicates that test on twelve typologically diverse languages from the Universal Dependencies treebank collection. The research question is narrow:

Research question: Do natural-language sentences have shorter total dependency length than randomized rearrangements of the same words under the same dependency tree, consistently across unrelated languages?

Objective: Measure, for each of twelve languages, whether the mean per-sentence dependency length under the natural word order is reliably smaller than under two random baselines, and estimate how large the difference is.

2. Hypotheses and predictions

H1 (main). In every language examined, the mean total dependency length of natural sentences is less than that of random rearrangements of the same words preserving the same dependency tree.

H2 (stronger test). The gap in H1 remains when the random baseline is restricted to projective rearrangements — those with no crossing dependency edges — showing that DLM operates beyond the no-crossing property itself.

Predictions if H1 and H2 hold: (i) for every language, the paired per-sentence difference (natural minus randomized) is negative on average; (ii) a paired one-sided test rejects the null at conventional levels ($\alpha = 0.05$) for every language; (iii) the effect appears across different dominant word-order types (SVO, SOV, VSO, free).

3. Methods

3.1 Data

Training sections of Universal Dependencies (UD) v2 treebanks (Nivre et al., 2020) were used for twelve languages spanning seven language families and four dominant word-order types (Table 1). Sentences of length 5 to 50 tokens were retained; multi-word tokens (e.g. French *du* = *de* + *le*) and empty nodes were skipped. From each treebank, 2,000 sentences were sampled at random, giving 24,000 sentences in total.

Table 1. Languages and UD treebanks used.

Language	Treebank	Word order	Family
English	UD_English-EWT	SVO	Indo-European (Germanic)
Hindi	UD_Hindi-HDTB	SOV	Indo-European (Indo-Aryan)
Japanese	UD_Japanese-GSD	SOV	Japonic
Arabic	UD_Arabic-PADT	VSO	Afro-Asiatic (Semitic)
Chinese	UD_Chinese-GSD	SVO	Sino-Tibetan
Russian	UD_Russian-SynTagRus	Free	Indo-European (Slavic)
Turkish	UD_Turkish-IMST	SOV	Turkic
Finnish	UD_Finnish-TDT	Free	Uralic
Korean	UD_Korean-GSD	SOV	Koreanic
Spanish	UD_Spanish-GSD	SVO	Indo-European (Romance)
Basque	UD_Basque-BDT	SOV	Language isolate
Indonesian	UD_Indonesian-GSD	SVO	Austronesian

3.2 Dependency length

For a sentence with n tokens, let $h(i)$ denote the position of the head of the token at position i , with $h(i) = 0$ for the root. For a linearization π (a permutation of token positions), the total dependency length is

$$D(\pi) = \sum_i |\pi(i) - \pi(h(i))| \text{ for all } i \text{ with } h(i) \neq 0.$$

For the natural sentence, π is the identity. For example, "The boy quickly opened the door." has edges the(1)→boy(2), boy(2)→opened(4), quickly(3)→opened(4), the(5)→door(6), door(6)→opened(4); $D = 1 + 2 + 1 + 1 + 2 = 7$. Rearranging to "Door quickly the opened boy the" while keeping the same edges gives $D = 5 + 3 + 2 + 4 + 4 = 18$.

3.3 Random baselines

Two baselines are used, each averaged over 10 independent draws per sentence:

(a) Random (unrestricted) — a uniformly random permutation of the n positions. The tree is preserved but the words are placed arbitrarily. Expected D grows quadratically with n .

(b) Random (projective) — a random linearization that keeps every subtree a contiguous span, forbidding crossing edges. Generated recursively: at each node, the slot order of the head and its children's subtrees is shuffled uniformly at random, and each child subtree is laid out in the chosen slot. This is the stronger baseline of Futrell et al. (2015): most natural sentences are themselves projective, so the projective baseline isolates the effect of word-order choice from the effect of structural well-formedness.

3.4 Statistical inference

For each language, natural D values are compared against each baseline using a paired one-sided Wilcoxon signed-rank test ($H_0: D_{\text{natural}} \geq D_{\text{baseline}}$; $H_1: D_{\text{natural}} < D_{\text{baseline}}$). The paired design matches each sentence with its own randomizations, so sentence length and tree structure are held constant. Wilcoxon is used rather than a paired t-test because D is right-skewed. Effect size is reported as the percentage reduction in mean D , and as the fraction of sentences in which the natural order is shorter than its own random baseline.

4. Results

H1 and H2 are supported in every one of the twelve languages. Figure 1 shows the average total dependency length per sentence under each condition. The natural word order (blue) is shorter than the projective random baseline (orange), which is shorter than the unrestricted random baseline (red), for every language.

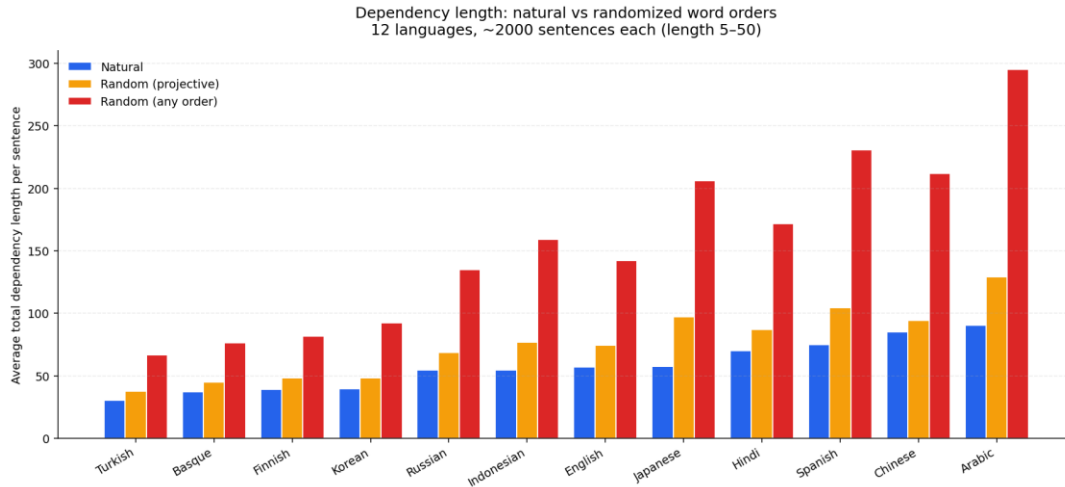


Figure 1. Average total dependency length per sentence, by language and condition (2,000 sentences per language).

Table 2. Mean dependency length per sentence and reductions vs each baseline.

Language	Natural	Projective	Random	% shorter vs Proj	% shorter vs Rand
Turkish	30.6	37.8	66.8	19.0%	54.1%
Basque	37.4	45.1	76.3	17.1%	51.0%
Finnish	39.0	48.1	81.5	19.0%	52.2%
Korean	39.6	48.2	92.6	17.9%	57.3%
Russian	54.6	68.7	134.9	20.5%	59.5%
Indonesian	54.7	76.7	159.3	28.6%	65.7%
English	57.2	74.3	142.4	23.0%	59.8%
Japanese	57.6	97.3	206.1	40.8%	72.0%
Hindi	70.0	87.0	171.7	19.5%	59.2%
Spanish	74.9	104.4	230.9	28.2%	67.5%
Chinese	85.3	94.2	212.0	9.5%	59.8%
Arabic	90.4	129.4	295.2	30.1%	69.4%

The reduction vs the unrestricted baseline ranges from 51% (Turkish) to 72% (Japanese). The reduction vs the stronger projective baseline ranges from 9.5% (Chinese) to 40.8% (Japanese). Paired one-sided Wilcoxon signed-rank tests are highly significant in every language, against both baselines (Table 3). Even in the weakest case (Chinese vs projective), 74.0% of individual sentences have shorter natural D than their own projective random counterpart.

Table 3. Sentence-level comparisons and one-sided Wilcoxon signed-rank p-values.

Language	% Nat < Rand	p (vs Rand)	% Nat < Proj	p (vs Proj)
Turkish	99.2%	< 1e-300	86.2%	2.20e-212
Basque	98.8%	< 1e-300	86.2%	6.42e-224
Finnish	96.0%	< 1e-300	79.5%	5.23e-179
Korean	98.1%	< 1e-300	83.5%	1.87e-198

Language	% Nat < Rand	p (vs Rand)	% Nat < Proj	p (vs Proj)
Russian	98.2%	< 1e-300	81.3%	1.34e-198
Indonesian	99.7%	< 1e-300	94.3%	4.77e-310
English	96.5%	< 1e-300	82.8%	1.82e-223
Japanese	99.6%	< 1e-300	98.4%	< 1e-300
Hindi	99.9%	< 1e-300	83.8%	8.20e-215
Spanish	99.5%	< 1e-300	91.3%	2.99e-295
Chinese	99.8%	< 1e-300	74.0%	2.64e-113
Arabic	99.2%	< 1e-300	90.0%	1.09e-279

The effect is not driven by short sentences. Figure 2 plots mean D against sentence length (pooled across languages): natural D grows roughly linearly with n, projective random D grows faster, and unrestricted random D grows almost quadratically, matching the theoretical $E[D] \approx n(n+1)/3$ for uniform permutations. The gap between natural and the baselines therefore widens as sentences get longer.

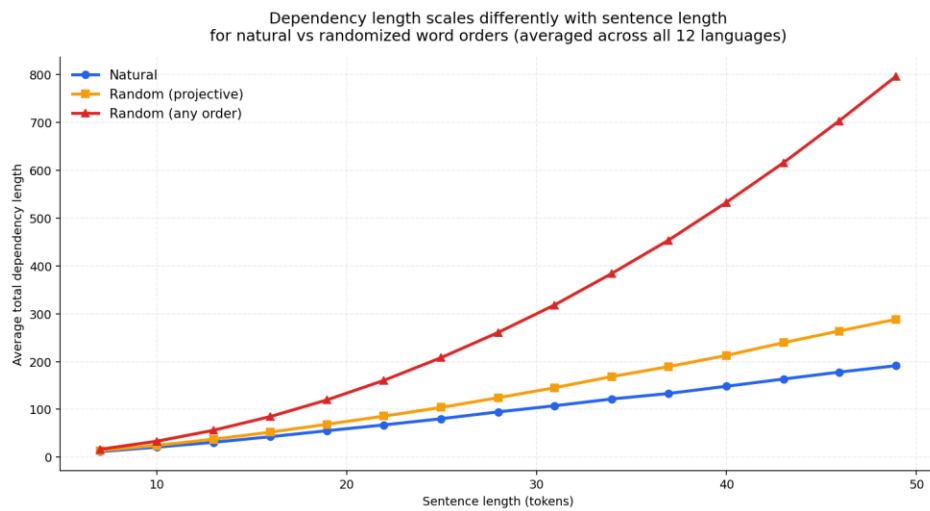


Figure 2. Average dependency length as a function of sentence length (pooled across 12 languages).

Grouping languages by dominant word-order type (SVO, SOV, VSO, free) preserves the same ordering natural < projective < random within every group (see Appendix A for the breakdown).

5. Theoretical implications

The consistent result across twelve unrelated languages supports a functional account of word order: human languages are shaped, at least in part, by the cognitive cost of producing and understanding sentences. Four specific implications follow.

Processing pressures are cross-linguistically universal. The effect appears in languages as different as Japanese (SOV, agglutinative, head-final) and Arabic (VSO, templatic, head-initial). A shared cognitive constraint — plausibly, working-memory limitations in parsing (Gibson, 1998) — is the simplest explanation for why structurally dissimilar languages converge on the same direction.

DLM operates over and above tree structure. The crucial test is against the projective baseline, not the unrestricted one. Real sentences beat even the random orderings that already preserve projectivity, so languages do not merely avoid crossing edges — among the projective orderings available, they actively favour the shorter

ones. Beating unrestricted random orders alone would be weak evidence, since any grammar that produces interpretable sentences does this.

Cross-language variation reflects competing pressures. The magnitude of the gap against the projective baseline varies from 40.8% (Japanese) to 9.5% (Chinese). One reading: languages with richer morphology and freer word order (Japanese, Russian) have more room to optimize within projective orderings, and their speakers exploit that freedom; languages with rigid word order and sparse morphology (Chinese) leave less room for further optimization because structural constraints already keep D low. This agrees with Liu, Xu & Liang (2017)'s broader observation that DLM interacts with morphological richness.

Implications for language evolution and for language models. DLM provides a concrete mechanism by which a cognitive pressure could shape grammar over generations of transmission: orderings that are easier to process are easier to learn and more likely to be reproduced (Christiansen & Chater, 2008). The fact that the effect is universal in direction but variable in magnitude fits this — languages converge on the same pressure but settle at different local optima because DLM competes with other demands (prosody, information density, discourse flow). Transformer-based language models, in contrast, have no architectural bias toward locality; whether they learn to produce DLM-compliant outputs from natural text, and whether evaluation should penalize non-local generations, is an empirical question raised by these results.

Limitations. The 2,000-sentence cap per language was chosen for tractability; treebank annotation quality varies; and dependency length is one of several proposed processing-cost metrics — intervening referents, integration cost, and surprisal all compete with or complement it. Addressing these in future work would sharpen, not overturn, the present conclusion.

References

- Christiansen, M. H., & Chater, N. (2008). Language as shaped by the brain. *Behavioral and Brain Sciences*, 31(5), 489–509.
- Futrell, R., Mahowald, K., & Gibson, E. (2015). Large-scale evidence of dependency length minimization in 37 languages. *Proceedings of the National Academy of Sciences*, 112(33), 10336–10341.
- Gibson, E. (1998). Linguistic complexity: Locality of syntactic dependencies. *Cognition*, 68(1), 1–76.
- Gibson, E. (2000). The Dependency Locality Theory: A distance-based theory of linguistic complexity. In A. Marantz, Y. Miyashita & W. O'Neil (eds.), *Image, Language, Brain*, pp. 95–126. MIT Press.
- Hawkins, J. A. (1994). *A Performance Theory of Order and Constituency*. Cambridge University Press.
- Liu, H., Xu, C., & Liang, J. (2017). Dependency distance: A new perspective on syntactic patterns in natural languages. *Physics of Life Reviews*, 21, 171–193.
- Nivre, J., et al. (2020). Universal Dependencies v2: An evergrowing multilingual treebank collection. *Proceedings of LREC*, 4034–4043.

Appendix A. Additional figures

Figure A1. Per-language distributions of dependency length per token.

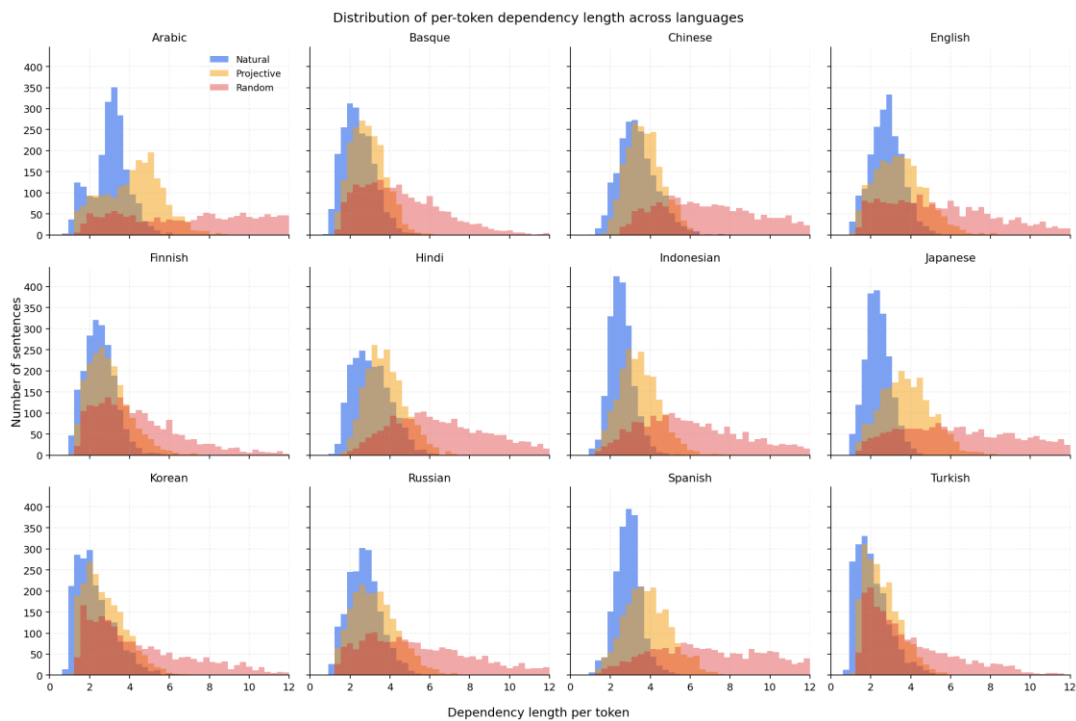


Figure A1. Distribution of dependency length per token, by language and condition. The natural (blue) distribution is consistently the leftmost — most concentrated at low values.

Figure A2. Average dependency length grouped by dominant word-order type.

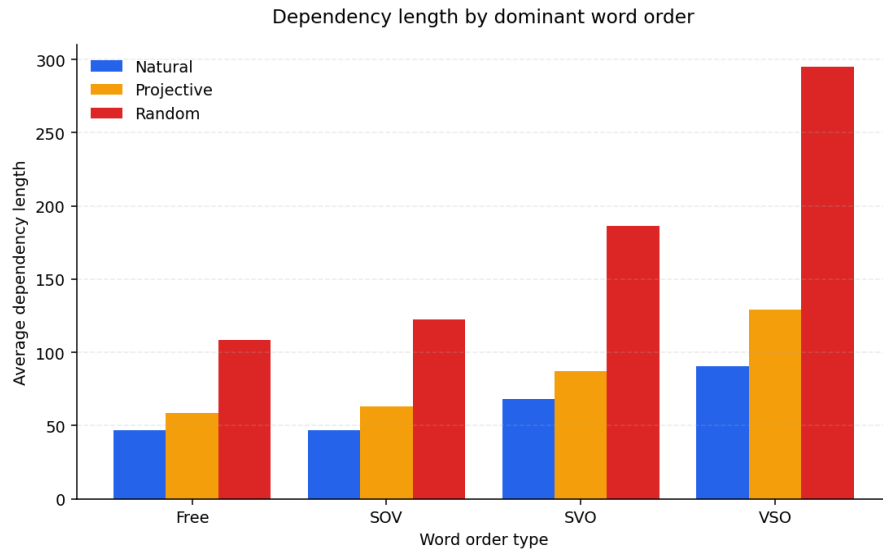


Figure A2. Dependency length by dominant word-order type. The ordering natural < projective < random holds within every group.

Figure A3. Percentage reduction vs each baseline.

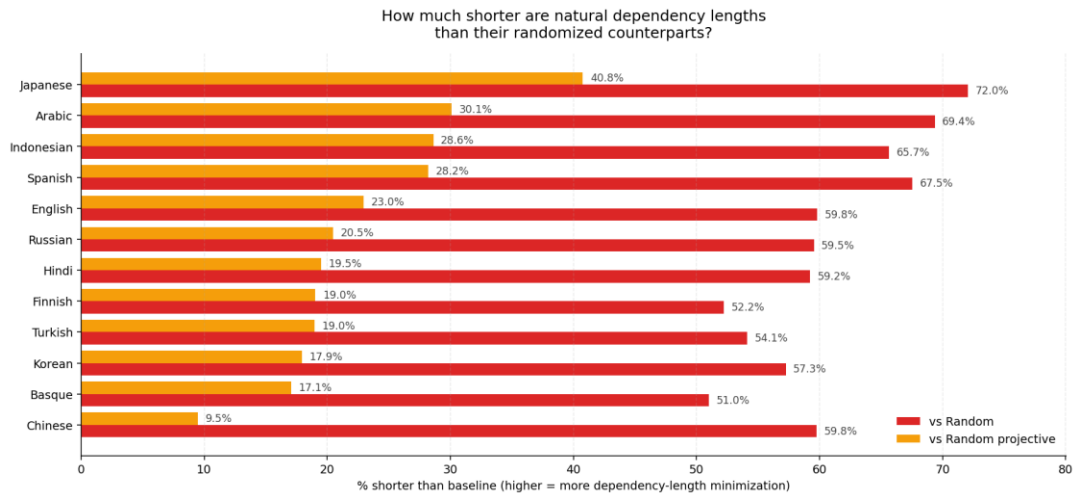


Figure A3. Percentage reduction of natural DL relative to each baseline, per language.

Appendix B. Code

The full implementation is also available on request. Below are the two Python scripts used to produce the results in the report. **analyze.py** downloads, parses and analyses the twelve UD treebanks; **visualize.py** produces the figures and runs the statistical tests.

B.1 analyze.py

```
"""
Dependency Length Analysis: Natural vs Randomized Word Orders
Across 12 typologically diverse languages from Universal Dependencies.

For each sentence:
- Natural DL = sum of |i - head(i)| in the observed word order
- Random DL = same, but with uniformly random word order
- Projective Random DL = random order preserving projectivity

Hypothesis: Natural < Random Projective < Random
"""

import conllu
import random
import numpy as np
import pandas as pd
from pathlib import Path
from collections import defaultdict

random.seed(42)
np.random.seed(42)

# ---- Configuration ----
DATA_DIR = Path("/home/claude/ud_data")
LANGUAGES = [
    # (Language, treebank_dir, train_file, word_order, family)
    ("English", "UD_English-EWT", "en_ewt-ud-train.conllu", "SVO", "Indo-European"),
    ("Hindi", "UD_Hindi-HDTB", "hi_hdtb-ud-train.conllu", "SOV", "Indo-European"),
    ("Japanese", "UD_Japanese-GSD", "ja_gsd-ud-train.conllu", "SOV", "Japonic"),
    ("Arabic", "UD_Arabic-PADT", "ar_padt-ud-train.conllu", "VSO", "Afro-Asiatic"),
    ("Chinese", "UD_Chinese-GSD", "zh_gsd-ud-train.conllu", "SVO", "Sino-Tibetan"),
    ("Russian", "UD_Russian-SynTagRus", "ru_syntagrus-ud-train-a.conllu", "Free", "Indo-European"),
    ("Turkish", "UD_Turkish-IMST", "tr_imst-ud-train.conllu", "SOV", "Turkic"),
    ("Finnish", "UD_Finnish-TDT", "fi_tdt-ud-train.conllu", "Free", "Uralic"),
    ("Korean", "UD_Korean-GSD", "ko_gsd-ud-train.conllu", "SOV", "Koreanic"),
    ("Spanish", "UD_Spanish-GSD", "es_gsd-ud-train.conllu", "SVO", "Indo-European"),
    ("Basque", "UD_Basque-BDT", "eu_bdt-ud-train.conllu", "SOV", "Isolate"),
    ("Indonesian", "UD_Indonesian-GSD", "id_gsd-ud-train.conllu", "SVO", "Austronesian"),
]

MAX_SENTENCES = 2000 # cap per language for tractability
MIN_LEN, MAX_LEN = 5, 50 # filter sentence length
N_RANDOM = 10 # # of random shuffles to average per sentence
N_PROJECTIVE = 10 # # of projective shuffles to average per sentence

# ---- Sentence extraction ----
def extract_sentence(tokens):
    """
    Return a list of (id, head) pairs from a parsed CoNLL-U sentence.
    Skips multiword tokens (id like '3-4') and empty nodes (id like '3.1').
    Returns None if any structural problem (missing head, etc.).
    """
    pairs = []
    for tok in tokens:
        tid = tok["id"]
        # Only keep simple integer ids (real words)
        if not isinstance(tid, int):
            continue
        head = tok["head"]
        if head is None:
            return None
        pairs.append((tid, head))
    return pairs

def dependency_length(pairs, position_map=None):
    """
    Compute total dependency length.
    pairs: list of (id, head) where id and head are 1-indexed positions in original sentence.
    """
```

```

position_map: dict {original_id -> new_position}. If None, use original positions.
Root dependencies (head==0) are excluded.
"""
total = 0
for tid, head in pairs:
    if head == 0:
        continue
    if position_map is None:
        total += abs(tid - head)
    else:
        total += abs(position_map[tid] - position_map[head])
return total

# ---- Random shuffling ----
def random_position_map(n):
    """Uniform random permutation: original id i -> new position perm[i-1]+1"""
    perm = list(range(1, n + 1))
    random.shuffle(perm)
    return {i + 1: perm[i] for i in range(n)}

# ---- Projective random shuffling ----
def build_children(pairs):
    """Build dict of head -> list of children (including root=0)."""
    children = defaultdict(list)
    for tid, head in pairs:
        children[head].append(tid)
    return children

def projective_random_layout(pairs):
    """
    Generate a random projective linearization of the dependency tree.

    Algorithm: At each node, randomly permute (node + its children's subtrees)
    and recursively layout each subtree as a contiguous span.

    Returns: position_map {original_id -> new_position}
    """
    children = build_children(pairs)
    roots = children[0] # nodes whose head is 0

    position_map = {}
    counter = [1] # mutable counter for next position

    def layout(node):
        # Children of `node` plus `node` itself need to be ordered.
        # Each is a "slot": either the head word (size 1) or a child subtree.
        slots = ['HEAD'] + list(children.get(node, []))
        random.shuffle(slots)
        for s in slots:
            if s == 'HEAD':
                position_map[node] = counter[0]
                counter[0] += 1
            else:
                layout(s)

    # Layout each root tree (most sentences have exactly one root)
    random.shuffle(roots)
    for r in roots:
        layout(r)

    return position_map

# ---- Per-sentence analysis ----
def analyze_sentence(pairs):
    """Return dict with natural DL and averaged random/projective DLs."""
    n = len(pairs)
    natural = dependency_length(pairs)

    rand_dls = [dependency_length(pairs, random_position_map(n)) for _ in range(N_RANDOM)]
    proj_dls = [dependency_length(pairs, projective_random_layout(pairs)) for _ in range(N_PROJECTIVE)]

    return {
        "n": n,
        "natural": natural,
        "random": np.mean(rand_dls),
        "projective": np.mean(proj_dls),
    }

```

```

# ---- Per-language analysis ----
def analyze_language(name, path, word_order, family):
    print(f" Processing {name} ({path.name})...")
    with open(path, encoding="utf-8") as f:
        sentences = list(conllu.parse_incr(f))

    # Optionally subsample BEFORE doing heavy work
    random.shuffle(sentences)

    results = []
    for sent in sentences:
        pairs = extract_sentence(sent)
        if pairs is None:
            continue
        n = len(pairs)
        if n < MIN_LEN or n > MAX_LEN:
            continue
        results.append(analyze_sentence(pairs))
        if len(results) >= MAX_SENTENCES:
            break

    df = pd.DataFrame(results)
    df["language"] = name
    df["word_order"] = word_order
    df["family"] = family
    print(f" -> {len(df)} sentences analyzed")
    return df

# ---- Main ----
def main():
    print("Loading and analyzing treebanks...")
    all_dfs = []
    for name, repo, fname, wo, fam in LANGUAGES:
        path = DATA_DIR / repo / fname
        if not path.exists():
            print(f" Skipping {name}: {path} not found")
            continue
        all_dfs.append(analyze_language(name, path, wo, fam))

    full = pd.concat(all_dfs, ignore_index=True)
    full.to_csv("/home/claude/sentence_results.csv", index=False)
    print(f"\nSaved {len(full)} sentence results to sentence_results.csv")

    # Per-language summary (means of per-sentence DL)
    summary = (full.groupby(["language", "word_order", "family"])
               .agg(n_sentences=("n", "count"),
                    avg_len=("n", "mean"),
                    natural=("natural", "mean"),
                    random=("random", "mean"),
                    projective=("projective", "mean"))
               .reset_index())
    summary["nat_vs_rand_pct"] = (1 - summary["natural"] / summary["random"]) * 100
    summary["nat_vs_proj_pct"] = (1 - summary["natural"] / summary["projective"]) * 100
    summary.to_csv("/home/claude/language_summary.csv", index=False)

    print("\n=== Per-language summary ===")
    print(summary.to_string(index=False))

    return full, summary

if __name__ == "__main__":
    main()

```

B.2 visualize.py

```

"""
Statistical tests + plots for the dependency-length comparison.
Reads sentence_results.csv produced by analyze.py.
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from scipy import stats
from pathlib import Path

```

```

plt.rcParams.update({
    "font.family": "DejaVu Sans",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "figure.dpi": 110,
})

NATURAL_C, PROJ_C, RAND_C = "#2563eb", "#f59e0b", "#dc2626"

OUT = Path("/mnt/user-data/outputs")
OUT.mkdir(parents=True, exist_ok=True)

df = pd.read_csv("/home/claude/sentence_results.csv")

# ----- 1. Statistical tests -----
print("=" * 78)
print("STATISTICAL TESTS (paired Wilcoxon signed-rank, per language)")
print("=" * 78)
print(f"{'Language':<12} {'n':>5} {'Nat<Rand %':>12} {'p (vs Rand)':>14} "
      f"{'Nat<Proj %':>12} {'p (vs Proj)':>14}")
print("-" * 78)

stats_rows = []
for lang in sorted(df["language"].unique()):
    sub = df[df["language"] == lang]
    nat, rand, proj = sub["natural"].values, sub["random"].values, sub["projective"].values

    # Sentence-level comparison: what fraction has natural < randomized?
    pct_nat_below_rand = (nat < rand).mean() * 100
    pct_nat_below_proj = (nat < proj).mean() * 100

    # Wilcoxon signed-rank (one-sided: natural < other)
    w_rand, p_rand = stats.wilcoxon(nat, rand, alternative="less")
    w_proj, p_proj = stats.wilcoxon(nat, proj, alternative="less")

    print(f"{'lang':<12} {'len(sub)':>5} {'pct_nat_below_rand':>11.1f}% "
          f"{'p_rand':>14.2e} {'pct_nat_below_proj':>11.1f}% {'p_proj':>14.2e}")

    stats_rows.append({
        "language": lang,
        "n_sentences": len(sub),
        "pct_nat_below_random": pct_nat_below_rand,
        "pct_nat_below_projective": pct_nat_below_proj,
        "p_value_vs_random": p_rand,
        "p_value_vs_projective": p_proj,
    })

pd.DataFrame(stats_rows).to_csv("/home/claude/statistical_tests.csv", index=False)

# ----- 2. Bar chart: average DL per language -----
summary = pd.read_csv("/home/claude/language_summary.csv").sort_values("natural")

fig, ax = plt.subplots(figsize=(13, 6))
x = np.arange(len(summary))
w = 0.27

ax.bar(x - w, summary["natural"], w, label="Natural", color=NATURAL_C, edgecolor="white")
ax.bar(x, summary["projective"], w, label="Random (projective)", color=PROJ_C, edgecolor="white")
ax.bar(x + w, summary["random"], w, label="Random (any order)", color=RAND_C, edgecolor="white")

ax.set_xticks(x)
ax.set_xticklabels(summary["language"], rotation=30, ha="right")
ax.set_ylabel("Average total dependency length per sentence")
ax.set_title("Dependency length: natural vs randomized word orders\n12 languages, ~2000 sentences each (length 5-50)", fontsize=13, pad=14)
ax.legend(loc="upper left", frameon=False)
ax.grid(axis="y", alpha=0.25, linestyle="--")
plt.tight_layout()
plt.savefig(OUT / "01_avg_dl_by_language.png", dpi=140, bbox_inches="tight")
plt.close()
print("\nSaved 01_avg_dl_by_language.png")

# ----- 3. % reduction (natural vs each baseline) -----
fig, ax = plt.subplots(figsize=(13, 6))
summary_s = summary.sort_values("nat_vs_proj_pct", ascending=True)
x = np.arange(len(summary_s))

ax.barh(x - 0.2, summary_s["nat_vs_rand_pct"], 0.4, label="vs Random", color=RAND_C)
ax.barh(x + 0.2, summary_s["nat_vs_proj_pct"], 0.4, label="vs Random projective", color=PROJ_C)

```

```

for i, (r, p) in enumerate(zip(summary_s["nat_vs_rand_pct"], summary_s["nat_vs_proj_pct"])):
    ax.text(r + 0.6, i - 0.2, f"{r:.1f}%", va="center", fontsize=9, color="#444")
    ax.text(p + 0.6, i + 0.2, f"{p:.1f}%", va="center", fontsize=9, color="#444")

ax.set_yticks(x)
ax.set_yticklabels(summary_s["language"])
ax.set_xlabel("% shorter than baseline (higher = more dependency-length minimization)")
ax.set_title("How much shorter are natural dependency lengths\nthan their randomized counterparts?", fontsize=13, pad=14)
ax.legend(loc="lower right", frameon=False)
ax.grid(axis="x", alpha=0.25, linestyle="--")
ax.set_xlim(0, max(summary_s["nat_vs_rand_pct"]) + 8)
plt.tight_layout()
plt.savefig(OUT / "02_pct_reduction.png", dpi=140, bbox_inches="tight")
plt.close()
print("Saved 02_pct_reduction.png")

# ----- 4. DL vs sentence length (averaged in length bins) -----
df["len_bin"] = pd.cut(df["n"], bins=range(5, 51, 3))
binned = df.groupby("len_bin", observed=True).agg(
    n_mid=("n", "mean"),
    natural=("natural", "mean"),
    projective=("projective", "mean"),
    random=("random", "mean"),
).reset_index()

fig, ax = plt.subplots(figsize=(11, 6))
ax.plot(binned["n_mid"], binned["natural"], marker="o", lw=2.5, color=NATURAL_C, label="Natural")
ax.plot(binned["n_mid"], binned["projective"], marker="s", lw=2.5, color=PROJ_C, label="Random (projective)")
ax.plot(binned["n_mid"], binned["random"], marker="^", lw=2.5, color=RAND_C, label="Random (any order)")
ax.set_xlabel("Sentence length (tokens)")
ax.set_ylabel("Average total dependency length")
ax.set_title("Dependency length scales differently with sentence length\nfor natural vs randomized word orders (averaged a
cross all 12 languages)", fontsize=13, pad=14)
ax.legend(frameon=False, fontsize=11)
ax.grid(alpha=0.25, linestyle="--")
plt.tight_layout()
plt.savefig(OUT / "03_dl_vs_length.png", dpi=140, bbox_inches="tight")
plt.close()
print("Saved 03_dl_vs_length.png")

# ----- 5. Per-language sentence-length-normalized DL distributions -----
df["natural_norm"] = df["natural"] / df["n"]
df["projective_norm"] = df["projective"] / df["n"]
df["random_norm"] = df["random"] / df["n"]

langs = sorted(df["language"].unique())
fig, axes = plt.subplots(3, 4, figsize=(15, 10), sharex=True, sharey=True)
axes = axes.flatten()

for i, lang in enumerate(langs):
    ax = axes[i]
    sub = df[df["language"] == lang]
    bins = np.linspace(0, 12, 40)
    ax.hist(sub["natural_norm"], bins=bins, alpha=0.6, color=NATURAL_C, label="Natural")
    ax.hist(sub["projective_norm"], bins=bins, alpha=0.5, color=PROJ_C, label="Projective")
    ax.hist(sub["random_norm"], bins=bins, alpha=0.4, color=RAND_C, label="Random")
    ax.set_title(lang, fontsize=11)
    ax.set_xlim(0, 12)
    ax.grid(alpha=0.2, linestyle="--")

axes[0].legend(loc="upper right", fontsize=9, frameon=False)
fig.supxlabel("Dependency length per token", fontsize=12)
fig.supylabel("Number of sentences", fontsize=12)
fig.suptitle("Distribution of per-token dependency length across languages", fontsize=13)
plt.tight_layout()
plt.savefig(OUT / "04_distributions.png", dpi=140, bbox_inches="tight")
plt.close()
print("Saved 04_distributions.png")

# ----- 6. Word-order grouping -----
wo_summary = summary.groupby("word_order")[["natural", "projective", "random"]].mean().reset_index()
fig, ax = plt.subplots(figsize=(8, 5))
x = np.arange(len(wo_summary)); w = 0.27
ax.bar(x - w, wo_summary["natural"], w, label="Natural", color=NATURAL_C)
ax.bar(x, wo_summary["projective"], w, label="Projective", color=PROJ_C)
ax.bar(x + w, wo_summary["random"], w, label="Random", color=RAND_C)
ax.set_xticks(x)

```

```

ax.set_xticklabels(wo_summary["word_order"])
ax.set_xlabel("Word order type")
ax.set_ylabel("Average dependency length")
ax.set_title("Dependency length by dominant word order", fontsize=12, pad=14)
ax.legend(frameon=False)
ax.grid(axis="y", alpha=0.25, linestyle="--")
plt.tight_layout()
plt.savefig(OUT / "05_by_word_order.png", dpi=140, bbox_inches="tight")
plt.close()
print("Saved 05_by_word_order.png")

print("\nDone.")

```